ESCOLA Máximu's
INFANTIL • FUNDAMENTAL • MÉDIO • TÉCNICO

GUIA PRÁTICO / PASSO A PASSO

Java + MySQL
no Eclipse

Da instalação à primeira conexão.
Do primeiro cadastro ao CRUD completo.

01 Prepare o ambiente

02 Conecte com JDBC

03 Cadastre e consulte dados

Prof. Bruno Michel Pera

Java 17+ • MySQL 8.4 • Connector/J 9.7.0

CÓDIGOS + 10 EXERCÍCIOS: github.com/perabru/java-mysql

01 / PREPARAÇÃO

Baixe as ferramentas

Links oficiais clicáveis para preparar o laboratório.

Você vai construir um cadastro de alunos no console. Primeiro execute a conexão; depois avance para inserir, listar, alterar e excluir registros.

JDK 21 LTS
https://adoptium.net/temurin/releases/?version=21

Escolha JDK e o seu sistema operacional. O código exige Java 17 ou superior.

Eclipse IDE
https://www.eclipse.org/downloads/packages/

Escolha Eclipse IDE for Java Developers. Mantenha o runtime exigido pela IDE.

MySQL Community Server 8.4 LTS
https://dev.mysql.com/downloads/mysql/8.4.html

O servidor armazena os dados. No Windows, prefira o instalador MSI.

MySQL Workbench
https://dev.mysql.com/downloads/workbench/

Editor visual para executar SQL; é opcional e não substitui o servidor.

MySQL Connector/J
https://dev.mysql.com/downloads/connector/j/

Driver JDBC. No projeto Maven ele é baixado automaticamente.

Connector/J 9.7.0 - arquivo oficial
https://downloads.mysql.com/archives/c-j/

Para instalação manual: versão 9.7.0, Platform Independent, ZIP.

> **Versões reproduzíveis:** o projeto fixa o driver 9.7.0. As páginas podem oferecer versões mais recentes; os menus e nomes dos instaladores também podem mudar.

02 / INSTALAÇÃO

Prepare o computador

Roteiro principal: Windows, Eclipse e MySQL local.

01 Instale o JDK

Marque a opção de adicionar ao PATH e JAVA_HOME, quando oferecida. Abra um terminal novo e confira:

```
java -version
javac -version
```

Os dois comandos devem indicar Java 17 ou superior. O JDK compila e executa; instalar apenas uma JRE não basta para desenvolver.

02 Abra o Eclipse

Instale Eclipse IDE for Java Developers, inicie a IDE e escolha uma pasta de workspace. Se o instalador trouxer um runtime próprio, mantenha-o para executar a IDE.

03 Instale o MySQL Server

Conclua o MySQL Configurator. Use porta **3306**, guarde a senha do administrador **root** e deixe a autenticação padrão e TLS habilitados. Configure o serviço para iniciar com o Windows.

04 Abra o Workbench

Crie ou abra uma conexão com host **localhost**, porta **3306**, usuário **root** e a senha criada. Clique em **Test Connection**.

> **Se não conectar:** pressione Win + R, digite **services.msc** e confira o serviço MySQL. O nome pode variar, por exemplo MySQL84. O Workbench aberto não significa que o servidor esteja ligado.

Entenda o caminho dos dados

Eclipse executa o programa → **JDBC / Connector/J** abre a comunicação → **MySQL Server** guarda e consulta os dados. O Workbench permite conferir os resultados usando SQL.

03 / BANCO DE DADOS

Crie banco e tabela

No Workbench, conectado como administrador root.

01 Abra o script

Baixe e extraia o projeto. Use **File → Open SQL Script** para abrir **sql/01_banco.sql**. Execute o script inteiro com o botão de raio correspondente.

```
-- Execute como administrador do MySQL. Nao apaga registros existentes.
CREATE DATABASE IF NOT EXISTS escola_maximus
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE escola_maximus;
CREATE TABLE IF NOT EXISTS alunos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);
```

02 Confira a estrutura

Atualize a área **Schemas → Refresh All**. Expanda escola_maximus → Tables → alunos. Execute:

```
USE escola_maximus;
SHOW TABLES;
SELECT * FROM alunos;
```

Uma lista vazia é normal: ainda não cadastramos alunos. O script cria a estrutura sem apagar registros existentes.

O que cada campo significa

id: número inteiro gerado automaticamente.
nome: texto obrigatório, até 100 caracteres.
email: texto obrigatório, até 150 caracteres, sem repetição.

> **Atenção ao banco selecionado:** o Java procurará a tabela no banco **escola_maximus**. Usar outro nome no SQL exige ajustar a URL de conexão.

04 / ACESSO

Crie o usuário da aula

O programa usará aluno_java; o administrador cria a estrutura.

01 Execute o segundo script

Ainda como root no Workbench, abra e execute **sql/02_usuario_local.sql**:

```
CREATE USER IF NOT EXISTS 'aluno_java'@'localhost'
  IDENTIFIED BY 'MaximusLocal_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON escola_maximus.*
  TO 'aluno_java'@'localhost';
```

Esse usuário pode consultar e modificar dados apenas no banco da aula. Ele não precisa de permissão para criar ou apagar bancos.

> **Senha didática e pública:** MaximusLocal_2026! serve apenas para este laboratório local. Use uma senha própria em ambientes reais e mantenha suas credenciais fora do GitHub.

02 Se o usuário já existir

CREATE USER IF NOT EXISTS preserva a senha anterior. Para mudar a senha conscientemente, execute como administrador:

```
ALTER USER 'aluno_java'@'localhost'
  IDENTIFIED BY 'SUA_NOVA_SENHA';
```

Depois coloque a mesma senha no arquivo **db.properties**. Não é preciso executar ALTER USER se esta for a primeira criação e você mantiver a senha da aula.

Teste de conferência

No Workbench, crie outra conexão usando **aluno_java** e a senha configurada. Test Connection deve funcionar. Volte à conexão root para tarefas de administração.

05 / ECLIPSE

Importe o projeto pronto

Caminho recomendado: Maven cuida do driver automaticamente.

Baixar todos os códigos em ZIP
https://github.com/perabru/java-mysql/archive/refs/heads/main.zip

01 Extraia o ZIP
Escolha uma pasta permanente. Identifique a pasta que contém **pom.xml**, README.md, sql e src.

02 Importe pelo Maven
No Eclipse: **File → Import → Maven → Existing Maven Projects → Next**. Em Root Directory selecione essa pasta, marque o projeto e clique em Finish.

03 Aguarde as dependências
A primeira importação precisa de internet. O Eclipse deve mostrar **Maven Dependencies**. Se falhar: botão direito no projeto → Maven → Update Project → Force Update → OK.

04 Confira o Java
Em **Window → Preferences → Java → Installed JREs**, selecione seu JDK. O projeto usa Java 17 como nível de compilação e pode rodar no JDK 21.

Dependência já incluída no pom.xml

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.7.0</version>
</dependency>
```

Maven baixa o Connector/J e inclui o driver no caminho de execução. Neste caminho você não precisa adicionar o JAR manualmente.

> **Quer digitar tudo em Java Project tradicional?** Siga a alternativa manual da página 13. Escolha um caminho por projeto para evitar drivers duplicados.

06 / CONFIGURAÇÃO

Configure a conexão

Um arquivo local mantém os dados de acesso fora do código Java.

01 Crie a cópia local

Na raiz, copie **db.properties.example** e renomeie a cópia para **db.properties**. Ela deve ficar ao lado do pom.xml, fora de src e sem .txt no final.

```
db.url=jdbc:mysql://localhost:3306/escola_maximus?\
  sslMode=REQUIRED&connectTimeout=5000&socketTimeout=10000
db.usuario=aluno_java
db.senha=MaximusLocal_2026!
```

No exemplo acima, a barra invertida no final da linha continua o valor na próxima linha. O arquivo do repositório já traz a URL em uma linha só.

02 Use os dados que você criou

Se alterou a senha ou a porta no MySQL, ajuste este arquivo. **localhost** significa o próprio computador; **3306** é a porta; **escola_maximus** é o banco.

03 Confira a pasta de execução

No Eclipse: **Run → Run Configurations → Java Application → Arguments → Working directory**. Selecione a raiz do projeto quando necessário.

Por que sslMode=REQUIRED?

A URL exige comunicação criptografada e funciona com o TLS padrão do MySQL local. connectTimeout e socketTimeout limitam a espera por conexão e resposta, em milissegundos.

> **Servidor remoto:** use **sslMode=VERIFY_IDENTITY**, hostname correto e certificado/CA confiável. REQUIRED criptografa, mas não verifica a identidade do servidor. Não desative TLS para contornar problemas.

Git: db.properties está no .gitignore. O arquivo example é público e contém apenas a configuração de laboratório.

07 / CÓDIGO COMPLETO

A classe Conexao

Arquivo: src/main/java/br/com/maximus/Conexao.java

Abra a classe pronta ou crie o pacote **br.com.maximus** e a classe com o mesmo nome. Este é o código completo publicado no projeto.

```
package br.com.maximus;

import java.io.IOException;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class Conexao {
    public static Connection conectar() throws SQLException {
        Properties config = new Properties();
        Path arquivo = Path.of("db.properties");
        try (Reader leitor = Files.newBufferedReader(
                arquivo, StandardCharsets.UTF_8)) {
            config.load(leitor);
        } catch (IOException e) {
            throw new SQLException(
                "Copie db.properties.example para db.properties "
                + "na raiz e confira o diretorio de trabalho.", e);
        }
        String url = config.getProperty("db.url");
        String usuario = config.getProperty("db.usuario");
        String senha = config.getProperty("db.senha");
        if (url == null || usuario == null || senha == null) {
            throw new SQLException(
                "Preencha db.url, db.usuario e db.senha.");
        }
        return DriverManager.getConnection(url, usuario, senha);
    }
}
```

Properties lê as três chaves. **DriverManager** abre a conexão. **SQLException** permite que a tela de execução mostre o motivo de uma falha.

08 / PRIMEIRA EXECUÇÃO

Teste antes do cadastro

Arquivo completo: TestarConexao.java

```
package br.com.maximus;

import java.sql.Connection;
import java.sql.SQLException;

public class TestarConexao {
    public static void main(String[] args) {
        try (Connection conexao = Conexao.conectar()) {
            System.out.println("Conectado ao banco: "
                + conexao.getCatalog());
        } catch (SQLException e) {
            System.err.println("Falha: " + e.getMessage());
            System.exit(1);
        }
    }
}
```

01 Execute a classe

Botão direito dentro do editor → **Run As → Java Application**. Veja a aba Console na parte inferior.

```
Conectado ao banco: escola_maximus
```

02 Confira o resultado

Se apareceu a mensagem acima, o Java abriu uma conexão real. Se falhar, leia o motivo e consulte a página 14 antes de avançar.

O que o try faz?

O **try-with-resources**, escrito como try (...), fecha a conexão automaticamente. O bloco catch mostra o erro. Em caso de falha, este teste encerra com código 1.

> **Imports:** Ctrl + Shift + O organiza as importações no Eclipse. Não é necessário Class.forName: o Connector/J é descoberto automaticamente quando está nas dependências.

09 / INSERIR

Seu primeiro INSERT

Trecho de AlunoDAO.java: método inserir, com ID gerado pelo banco.

```
    public static int inserir(String nome, String email)
            throws SQLException {
        validar(nome, email);
        String sql = "INSERT INTO alunos (nome, email) VALUES (?, ?)";
        try (Connection c = Conexao.conectar();
             PreparedStatement ps = c.prepareStatement(
                 sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, nome.trim());
            ps.setString(2, email.trim());
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) return rs.getInt(1);
                throw new SQLException("O banco nao retornou o ID.");
            }
        }
    }
```

Leia na ordem

1. validar confere nome e e-mail.
2. O SQL define duas posições com **?**.
3. setString(1, ...) preenche nome e setString(2, ...) preenche e-mail.
4. executeUpdate executa a inserção.
5. getGeneratedKeys recupera o ID automático.

> **Por que usar parâmetros?** O texto digitado é tratado como dado. Nomes com apóstrofo funcionam sem montar SQL por concatenação. Os índices de parâmetros começam em 1.

Este método é um trecho da classe completa: use os imports e os métodos de validação já fornecidos no repositório. O menu chama AlunoDAO.inserir(nome, email).

10 / CONSULTAR E MODIFICAR

Complete o CRUD

Trechos de AlunoDAO.java. CRUD: criar, ler, atualizar e excluir.

```
public static List<String> listar() throws SQLException {
    List<String> alunos = new ArrayList<>();
    String sql = "SELECT id, nome, email FROM alunos ORDER BY id";
    try (Connection c = Conexao.conectar();
         PreparedStatement ps = c.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            alunos.add(rs.getInt("id") + " | "
                + rs.getString("nome") + " | "
                + rs.getString("email"));
        }
    }
    return alunos;
}

public static boolean atualizar(int id, String nome, String email)
        throws SQLException {
    validar(nome, email);
    validarId(id);
    String sql = "UPDATE alunos SET nome = ?, email = ? WHERE id = ?";
    try (Connection c = Conexao.conectar();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, nome.trim());
        ps.setString(2, email.trim());
        ps.setInt(3, id);
        return ps.executeUpdate() > 0;
    }
}

public static boolean excluir(int id) throws SQLException {
    validarId(id);
    String sql = "DELETE FROM alunos WHERE id = ?";
    try (Connection c = Conexao.conectar();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setInt(1, id);
        return ps.executeUpdate() > 0;
    }
}
```

SELECT: executeQuery retorna linhas e next avança o cursor. **UPDATE/DELETE:** executeUpdate informa as linhas afetadas. O WHERE limita a operação ao ID informado.

11 / PROGRAMA PRONTO

Use o cadastro

Execute CadastroAlunos.java como Java Application e digite no Console.

```
MAXIMUS | CADASTRO DE ALUNOS
1 Cadastrar | 2 Listar | 3 Alterar
4 Excluir   | 0 Sair
```

01 Cadastre
Digite 1, um nome fictício e um e-mail como ana@example.com. Anote o ID retornado pelo programa.

02 Liste
Digite 2. O resultado mostra ID, nome e e-mail. Se não houver alunos, o programa informa que a lista está vazia.

03 Altere
Digite 3, o ID e novos dados. Liste novamente. Se o ID não existir, o programa informa isso.

04 Exclua com confirmação
Digite 4, o ID e S para confirmar. Com N a exclusão é cancelada. Digite 0 para encerrar.

Confira no Workbench

```
SELECT * FROM escola_maximus.alunos;
```

Fechar o Java não apaga os dados. O menu trata e-mail duplicado, formato básico inválido, campo vazio e ID inválido sem interromper o uso normal.

Organização: CadastroAlunos lê o teclado; AlunoDAO executa SQL; Conexao abre o acesso ao banco. Todas as classes completas estão em src/main/java/br/com/maximus.

12 / ALTERNATIVA

Use JAR sem Maven

Para a aula em Java Project tradicional. Escolha este caminho desde o início.

01 Crie o projeto

File → New → Java Project. Nome: JavaMySQL. Use JDK 17 ou superior. Se perguntar sobre module-info.java, escolha **Don't Create**.

02 Crie o pacote e as classes

Dentro de src, crie br.com.maximus. Copie para ele as quatro classes Java do repositório: Conexao, TestarConexao, AlunoDAO e CadastroAlunos.

03 Baixe e extraia o driver

No arquivo oficial do Connector/J, escolha 9.7.0, Platform Independent e ZIP. Extraia o arquivo **mysql-connector-j-9.7.0.jar**.

Abrir arquivo oficial do Connector/J
https://downloads.mysql.com/archives/c-j/

04 Adicione ao Classpath

Crie uma pasta lib na raiz e coloque o JAR nela. Botão direito no projeto → Build Path → Configure Build Path → Libraries → **Classpath** → Add JARs. Selecione o arquivo → Apply and Close.

05 Configure e execute

Copie o example para a raiz do novo projeto como db.properties. Prepare os scripts SQL e execute TestarConexao, depois CadastroAlunos.

> **Sinal de sucesso:** o JAR aparece em Referenced Libraries. Se aparecer No suitable driver, confira se foi adicionado ao Classpath do projeto que você está executando.

O caminho Maven já configura tudo pelo pom.xml. Não é preciso misturar as duas formas. Em qualquer caminho, use o mesmo banco e as mesmas classes.

13 / DIAGNÓSTICO

Resolva os erros comuns

Leia a mensagem do Console: ela indica qual etapa precisa ser corrigida.

No suitable driver
Confira Maven Dependencies ou JAR no Classpath; a URL deve começar com jdbc:mysql:.

Access denied
Confira usuário, senha e script de permissão. Se o usuário já existia, a senha antiga foi preservada.

Unknown database / tabela ausente
Execute 01_banco.sql inteiro e confirme escola_maximus e alunos no Workbench.

Communications link failure
Confira o serviço MySQL, localhost e porta 3306. Não exponha a porta na internet.

Arquivo db.properties não encontrado
Confira raiz, extensão sem .txt e Working directory da execução.

Erro de TLS ou Public Key Retrieval
Confira o TLS padrão do servidor e a URL fornecida. Não acrescente parâmetros inseguros automaticamente.

E-mail já cadastrado
Use outro e-mail fictício ou altere o cadastro existente. O campo é único no banco.

UnsupportedClassVersionError
A execução está usando Java antigo. Selecione JDK 17 ou superior.

Acentos incorretos / erro de módulo
Use UTF-8 no projeto e Console; use este exemplo sem module-info.java.

Verificação automatizada opcional

Com Maven instalado e db.properties pronto, execute na raiz. O teste usa o MySQL real e remove o aluno temporário ao final.

```
mvn test-compile
mvn exec:java \
  -Dexec.mainClass=br.com.maximus.VerificarCRUD \
  -Dexec.classpathScope=test
```

No Windows, digite o último comando inteiro em uma linha, sem as barras de continuação. Esperado: APROVADO: 12 verificacoes. O GitHub Actions repete a verificação com MySQL 8.4.

14 / PRÁTICA

10 exercícios para fazer

Enunciados completos e critérios de entrega também estão no README do projeto.

01 Primeira conexão
Prepare o ambiente e execute TestarConexao. Explique host, porta e banco; não mostre a senha.

02 Três cadastros
Cadastre três alunos e confira a lista no Java e no Workbench. Os IDs devem ser diferentes.

03 Buscar por ID
Crie a opção 5 com WHERE id = ?. Mostre o aluno encontrado ou uma mensagem para ID ausente.

04 Pesquisar por nome
Use LIKE ? e ordene por nome. Teste uma busca com vários resultados e outra sem resultados.

05 Atualizar e-mail
Altere somente o e-mail por ID. Preserve o nome e trate e-mail inválido e duplicado.

06 Exclusão segura
Mostre os dados antes de excluir e peça S/N. Teste cancelamento, confirmação e ID ausente.

07 Contar alunos
Use SELECT COUNT(*). Confira o total antes e depois de cadastrar e em uma base de testes vazia.

08 Adicionar turma
Crie turma VARCHAR(20) com ALTER TABLE e adapte o CRUD. Preserve cadastros antigos.

09 Validar entradas
Rejeite nome com um caractere, e-mail vazio e ID negativo. Documente cinco testes sem encerrar o menu.

10 Mini-sistema de produtos
Faça CRUD de produtos com preço DECIMAL(10,2), BigDecimal e estoque INT. Rejeite valores negativos.

Entrega: código, SQL adicional e evidências de execução com dados fictícios. Use PreparedStatement e feche os recursos. O README detalha todos os testes esperados.

15 / REFERÊNCIAS

Continue no repositório

Uma base pequena para aprender conexão e persistência com clareza.

Projeto completo e README
https://github.com/perabru/java-mysql

Códigos em ZIP
https://github.com/perabru/java-mysql/archive/refs/heads/main.zip

Verificações no GitHub Actions
https://github.com/perabru/java-mysql/actions

Documentação oficial

Java: DriverManager
https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/DriverManager.html

Java: PreparedStatement
https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html

MySQL: instalação do Connector/J
https://dev.mysql.com/doc/connector-j/en/connector-j-installing.html

MySQL: conexão com TLS
https://dev.mysql.com/doc/connector-j/en/connector-j-reference-using-ssl.html

Antes de entregar sua atividade

Confira a execução da conexão, as quatro operações do CRUD, a persistência no Workbench e os casos de erro. Mantenha db.properties fora do Git. Utilize nomes e e-mails fictícios.

Material didático: Prof. Bruno Michel Pera • Escola Máximu's.
Referências consultadas em 20/10/2026. O logo e a paleta foram consultados no site da escola: escolamaximus.com.br. O PDF contém a conexão completa e trechos identificados do DAO; todas as classes completas estão no projeto.